Cloud Storage and Content Delivery Networks (CDN)

Types of cloud storage

a. File Storage in Cloud Computing

⇨ File Storage is a type of cloud storage where data is stored as files organized in folders and directories, similar to the file system used in a computer.

Working

⇨ Data is stored as individual files.
⇨ Files are organized into folders and subfolders.
⇨ Users access files through file names and paths.

Advantages

- Easy to understand and manage.
- Supports file sharing among users.
- Suitable for documents and multimedia files.
- Familiar folder-based structure.

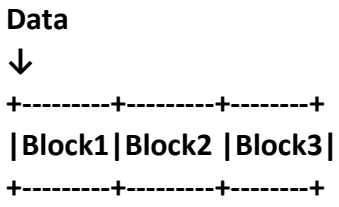
Disadvantages

- Performance decreases with very large datasets.
- Less scalable than object storage.
- Searching large numbers of files can be difficult.

b. Block Storage in Cloud Computing

⇨ Block Storage is a type of cloud storage where data is divided into fixed-size blocks, and each block is stored separately with a unique identifier.
⇨ The operating system combines these blocks to access the complete data.

Structure



Advantages

- High performance and low latency.
- Fast data access.
- Suitable for databases and virtual machines.
- Easy to modify individual blocks**.**

Disadvantages

- More complex management.
- Can be more expensive than file storage.

Applications

- Databases
- Virtual Machines (VMs)
- Enterprise applications
- High-performance workloads

c. Object Storage in Cloud Computing

⇨ Object Storage is a type of cloud storage where data is stored as objects. Each object contains:

I. **Data** (the actual file)
II. **Metadata** (information about the file)
III. **Unique Identifier (ID)**

Structure

Object
├─── Data
├─── Metadata
└------ Unique ID

Example:

Photo.jpg
├── Data (Image)
├── Metadata (Size, Date, Owner)
└── ID: 12345

Advantages

- Highly scalable.
- Stores massive amounts of unstructured data.
- Cost-effective.
- Easy data management through metadata.
- High durability and availability.

Disadvantages

- Higher latency than block storage.
- Not suitable for databases requiring frequent updates.
- Cannot be mounted like a traditional file system.

Applications

- Image and video storage
- Cloud backups
- Big data storage
- Social media content
- Cloud-native applications

Object Storage APIs

⇨ Object Storage APIs are application programming interfaces that allow applications to store, retrieve, update, and delete objects in object storage systems.

Main Operations of Object Storage APIs

1. **PUT**
   - Uploads an object to storage.
2. **GET**
   - Retrieves an object from storage.
3. **DELETE**
   - Removes an object from storage.
4. **LIST**
   - Displays available objects in a storage bucket.
5. **UPDATE**
   - Modifies object metadata or content.

Features

- REST-based access (HTTP/HTTPS)
- Easy integration with applications
- Scalable and reliable
- Supports metadata management
- Secure access through authentication

Advantages

- Simple access to cloud storage
- Platform independent
- Supports large-scale data storage
- Easy automation and integration

Example

An application uploads an image to cloud object storage using a **PUT** request and later retrieves it using a **GET** request.

Common object storage services provide APIs, such as:

- Amazon S3 API
- Google Cloud Storage API

Redundancy

⇨ Redundancy is the practice of maintaining duplicate copies of data, servers, or network components to ensure continuous service availability in case of failure.

Working

- Data is stored in multiple locations.
- If one server or storage device fails, another copy is used automatically.
- Ensures high availability and fault tolerance.

Advantages

- Prevents data loss
- Improves reliability
- Ensures continuous service
- Provides fault tolerance

Example: A cloud provider stores the same data on multiple servers in different data centers.

Backup

⇨ Backup is the process of creating and storing copies of data so that it can be restored if the original data is lost, damaged, or deleted.

Working

- Data is copied periodically.
- Backup copies are stored in a separate location.
- Data can be restored when needed.

Types of Backup

- **Full Backup** – Copies all data.
- **Incremental Backup** – Copies only changed data since the last backup.
- **Differential Backup** – Copies all changes since the last full backup.

Advantages

- Protects against accidental deletion
- Enables disaster recovery
- Helps restore corrupted data
- Ensures business continuity

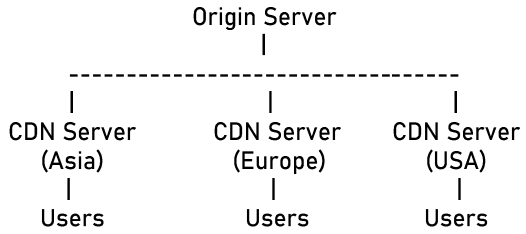
Example: A company automatically backs up its database every night to cloud storage.

Content Delivery Networks (CDNs)

⇨ A **Content Delivery Network (CDN)** is a network of geographically distributed servers that deliver web content to users from the server nearest to them.

⇨ The main goal of a CDN is to improve website performance, reduce latency, and increase availability.

Working of CDN



- Content is stored on the **Origin Server**.
- Copies of the content are cached on CDN servers worldwide.
- When a user requests content, it is delivered from the nearest CDN server.
- This reduces loading time and network traffic.

Features of CDN

- Faster content delivery
- Reduced latency
- Load balancing
- High availability
- Improved security

Advantages

1. Faster website loading.
2. Reduced server load.
3. Better user experience.
4. Increased reliability and availability.
5. Protection against DDoS attacks.

Applications

- Websites
- Video streaming services
- Online gaming
- E-commerce platforms
- Software downloads

Example: When a user in India accesses a website, the CDN delivers content from a nearby server in India instead of fetching it from a distant server in another country. (cloudflare)

Data Caching

⇨ **Data Caching** is the process of storing frequently accessed data in a temporary high-speed storage area called a **cache** so that it can be retrieved quickly.

Advantages

- Faster data access
- Reduced latency
- Lower database load
- Improved application performance

Data Replication

⇨ **Data Replication** is the process of creating and maintaining multiple copies of data at different locations or servers.

Working

1. Original data is stored on one server.
2. Copies of the data are created on other servers.
3. If one server fails, data can be accessed from another copy.

Advantages

- High availability
- Fault tolerance
- Disaster recovery
- Improved reliability

Example: A cloud provider stores the same data in multiple data centers to prevent data loss.

Cloud Database Services

- ⇨ **Cloud Database Services** are database systems hosted and managed by cloud providers. They allow users to store, manage, and access data over the Internet without maintaining physical database servers.
- ⇨ The cloud provider handles installation, maintenance, backups, updates, security, and scalability.

Features

- On-demand access
- Automatic backup and recovery
- Scalability
- High availability
- Security and encryption
- Managed maintenance

Types of Cloud Databases

1. **Relational Databases (SQL)**
   - Store data in tables (rows and columns).
   - Use Structured Query Language (SQL).

   Example
   i. Amazon RDS
   ii. Cloud SQL
2. **Non-Relational Databases (NoSQL)**
   - Store unstructured or semi-structured data.
   - Suitable for large-scale applications.

   Example
   i. MongoDB Atlas
   ii. Amazon DynamoDB

Advantages

- No hardware management.
- Easy scalability.
- Automatic backup and recovery.
- High availability and reliability.
- Cost-effective (pay-as-you-use).

Disadvantages

- Internet dependency.
- Security and privacy concerns.
- Vendor lock-in risk.
- Limited control compared to self-hosted databases.

Emerging Trends in Cloud Computing

Why Hybrid Cloud Is an Emerging Trend?

- Organizations need both security and scalability.
- Sensitive data remains in the private cloud.
- Public cloud is used for additional resources when demand increases.

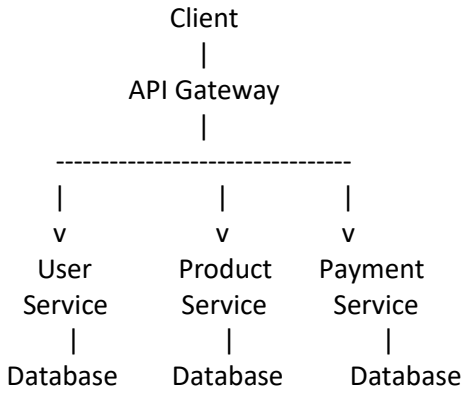
Why Multi-Cloud Is an Emerging Trend?

- Avoids dependence on a single provider (**vendor lock-in**).
- Allows organizations to choose the best services from different providers.
- Improves reliability and disaster recovery.

Microservices Architecture

⇨ **Microservices Architecture** is a software development approach in which an application is divided into a collection of small, independent services. Each service performs a specific business function and communicates with other services through APIs.

Architecture Diagram



Each microservice:

- Has its own logic
- Has its own database
- Can be developed and deployed independently

Working

- The application is divided into small services.
- Each service handles a specific task.
- Services communicate using APIs (REST, gRPC, etc.).
- Each service can be updated without affecting others.

Advantages

- Easy scalability
- Faster development and deployment
- Better fault isolation
- Technology flexibility
- Easier maintenance

Disadvantages

- Complex management
- Increased network communication
- Difficult monitoring and debugging
- Higher deployment complexity

Applications

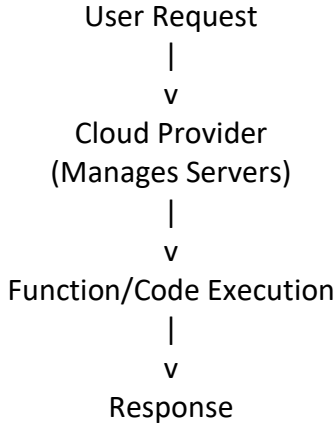
- E-commerce platforms
- Banking systems
- Streaming services
- Cloud-native applications

Serverless Computing

⇨ Serverless Computing is a cloud computing model where developers run code without managing servers, while the cloud provider automatically handles infrastructure, scaling, and maintenance.

⇨ Despite the name **"serverless"**, servers are still used, but they are managed by the cloud provider.

Architecture



Working

1. Developer uploads code (function).
2. A user or event triggers the function.
3. The cloud provider automatically allocates resources.
4. The function executes and returns the result.
5. Resources are released after execution.

Advantages

- No infrastructure management.
- Automatic scaling.
- Reduced operational cost.
- Faster development and deployment.
- Pay only for actual usage.

Disadvantages

- Cold start latency.
- Limited execution time.
- Less control over infrastructure.
- Vendor lock-in risk.

Example

A photo-upload application automatically resizes images whenever a user uploads a photo. The function runs only when triggered and stops after completing the task.

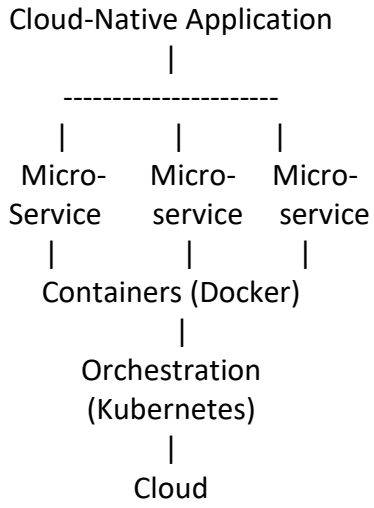
Popular Serverless platforms:

AWS Lambda
Google Cloud Functions

Cloud-Native Development

⇨ Cloud-Native Development is an approach to building and running applications designed for cloud environments using technologies such as microservices, containers, orchestration, and DevOps.

Architecture



Key Technologies

1. **Microservices** – Application divided into small independent services.
2. **Containers** – Package applications and dependencies together.
3. **Orchestration** – Manage containers automatically.
4. **DevOps & CI/CD** – Continuous development and deployment.
5. **Serverless Computing** – Run code without managing servers.

Advantages

- Faster application development.
- Easy scalability.
- High availability.
- Better resource utilization.
- Continuous delivery and updates.

Disadvantages

- Complex architecture.
- Requires skilled teams.
- Difficult monitoring and debugging.
- Security management challenges.

DevOps

⇨ **DevOps** on the Cloud is the practice of applying DevOps principles and tools in a cloud environment to automate software development, testing, deployment, and operations.

Key Components

1. **Continuous Integration (CI)**
   - Developers frequently merge code into a shared repository.
   - Code is automatically built and tested.
2. **Continuous Delivery/Deployment (CD)**
   - Applications are automatically deployed to the cloud after successful testing.
3. **Automation**
   - Automates testing, deployment, monitoring, and infrastructure management.
4. **Monitoring**
   - Continuously tracks application performance and system health.

Advantages

- Faster development and deployment.
- Improved software quality.
- Reduced operational costs.
- Better collaboration between teams.
- Rapid scalability using cloud resources.

Disadvantages

- Requires skilled professionals.
- Complex tool integration.
- Security and compliance challenges.

DevOps Tools : Docker, Kubernetes, Git

Continuous Integration (CI)

⇨ **Continuous Integration (CI)** is the practice of frequently merging code changes into a shared repository, where automated builds and tests are performed.

Steps

1. Developer writes code.
2. Code is pushed to a repository.
3. Automated build starts.
4. Automated tests run.
5. Errors are detected early.

Advantages

- Early bug detection
- Improved code quality
- Faster development
- Better team collaboration

Continuous Deployment (CD)

⇨ **Continuous Deployment (CD)** is the practice of automatically deploying tested code to production or staging environments.

Steps

1. Code passes all tests.
2. Application is automatically deployed.
3. Users receive updates quickly.

Advantages

- Faster software delivery
- Reduced manual effort
- Quick feedback from users
- Reliable deployments

Benefits of CI/CD

1. Faster development cycle.
2. Improved software quality.
3. Reduced deployment failures.
4. Automation of repetitive tasks.
5. Faster delivery of new features.

Application Migration

⇨ **Application Migration** is the process of moving an application, its data, and workloads from one environment to another, usually from on-premises infrastructure to the cloud.

Steps in Migration

1. Assess the existing application.
2. Plan the migration strategy.
3. Transfer application and data.
4. Test the application.
5. Deploy in the cloud.

Advantages

- Reduced infrastructure cost
- Improved scalability
- Better performance
- Increased availability

Application Modernization

⇨ **Application Modernization** is the process of updating old (legacy) applications using modern cloud technologies and architectures to improve performance, scalability, and maintainability.

Modernization Techniques

- Converting monolithic applications to microservices
- Using containers
- Adopting cloud-native development
- Implementing DevOps and CI/CD
- Using serverless computing

Advantages

- Better scalability
- Faster deployment
- Improved reliability
- Easier maintenance
- Enhanced user experience

Example

A legacy banking application is redesigned into microservices and deployed using containers in the cloud.